BABEŞ BOLYAI UNIVERSITY, CLUJ NAPOCA, ROMÂNIA FACULTY OF MATHEMATICS AND COMPUTER SCIENCE

Multimodel VS Code Extension

Team Members

Croitoru Andreea-Bianca
Group 232

Dobrescu Paul Andrei
Group 232

Moghioros Eric
Group 234

2025-2026

Contents

Local Multimodel Assistant for Visual Studio Code

1 Introduction

1.1 Context

The Multimodel AI Assistant project aims to develop an extension for the Visual Studio Code environment that integrates advanced intelligent assistance functionalities into the software development process. The main objective is to create a tool capable of supporting developers in activities such as code analysis and review, security vulnerability detection, project knowledge management, and workflow optimization through the orchestration of multiple artificial intelligence models.

The proposed system employs a multimodal architecture, in which several AI models, both local and cloud-based, collaborate to generate contextual and efficient responses. This approach enables dynamic adaptation of computational resources, performance optimization, and data confidentiality, depending on the specific characteristics of the analyzed project.

1.2 Motivation

In the current context of software development, the complexity of information systems has increased significantly, generating a growing need for tools that facilitate the automation of analysis and verification processes. Modern software projects involve thousands of lines

of code, geographically distributed teams, and strict quality and security requirements. Under these circumstances, traditional manual code review methods become inefficient and error-prone to humans.

The motivation for employing an intelligent algorithm within this project arises from the necessity of optimizing the development process through the following aspects:

- *Operational efficiency:* AI systems can automatically analyze extensive code segments, rapidly identifying errors, inconsistencies, or potential vulnerabilities.
- *Continuous and adaptive learning:* The system can accumulate knowledge from previous projects, adapting to the team's coding style and the specific domain of the application.
- *Enhanced software security:* Automated analysis modules can identify vulnerabilities that are difficult to detect through conventional methods, thus reducing security risks.
- *Data confidentiality:* Through support for local models, code analysis can be performed internally without exposing sensitive information to external services.

Therefore, the implementation of a solution based on intelligent algorithms represents an essential step toward modernizing the software development process, ensuring a balance between performance, security, and adaptability.

2 Contemporary Holdbacks

Artificial intelligence has emerged as a promising approach for automating parts of the software development process. Large language models (LLMs) and machine learning systems can analyze source code, generate documentation, and provide intelligent suggestions. However, current AI-powered tools often suffer from three critical limitations:

- *Single-model dependency:* Most tools rely on one large, centralized model, which limits flexibility and adaptability across diverse tasks.

- *Privacy and security concerns:* Cloud-based AI services require sending source code to external servers, raising confidentiality and compliance issues for organizations handling sensitive or proprietary data.
- *Lack of contextual learning:* Existing systems typically provide isolated, one-time analyses without learning from past interactions or adapting to the specific coding practices of a development team.

The scientific contribution lies in developing a framework that merges AI orchestration, knowledge-based adaptation, and privacy-aware computation into a unified assistant. This framework represents a new step toward intelligent software development environments, where artificial intelligence operates not as a single tool but as a coordinated ecosystem of models that understand, learn, and evolve with the developer's workflow.

3 Related Work

The development of intelligent software assistants has received increasing attention in recent years, driven by advances in natural language processing, machine learning, and software engineering tools. Several approaches have been proposed to support developers in tasks such as code review, vulnerability detection, documentation generation, and automated refactoring.

3.1 OpenAI Codex

- *Data:* The original OpenAI Codex model was trained on approximately 159 GB of Python code from over 54 million public GitHub repositories, covering multiple programming languages including JavaScript, Go, Ruby, and C++. The dataset included a range of software projects from simple scripts to complex applications, curated to ensure high quality and adherence to good programming practices. Reinforcement learning from human feedback (RLHF) was incorporated in later versions to improve autonomous coding capabilities and understand real-world coding scenarios.

- *Algorithm:* Codex is based on the GPT-3 architecture fine-tuned specifically for programming tasks. It operates by analyzing input prompts—such as natural language descriptions or partial code snippets—and predicting the most probable continuation of code tokens. The model uses statistical patterns learned from the training data to generate code but does not execute or logically reason about the code. Iterative prompt refinement helps improve the quality of generated code.

- *Outcomes:* Codex can complete approximately 37% of coding requests accurately and generates working solutions for around 70% of test prompts when sampled multiple times. It supports over a dozen programming languages with highest performance in Python. Codex accelerates programming by generating code snippets, automating tasks like opening pull requests, refactoring files, and writing tests. Its limitations include occasional errors and dependence on prompt quality, making it a tool to assist rather than replace human developers.

3.2 Rethinking Code Review Workflows with LLM Assistance

- *Data:* The study used empirical data from an industrial field study conducted at WirelessCar Sweden AB, comprising semi-structured interviews and real-world experiments with two prototype LLM-assisted code review tools. The tools integrated retrieval-augmented generation (RAG) to provide contextual information during code reviews, addressing challenges like frequent context switching and insufficient context.

- *Algorithm:* Two variations of LLM-assisted review tools were developed: one providing upfront LLM-generated reviews (AI-led co-reviewer) and another offering on-demand interaction with the LLM (interactive assistant). Both leveraged semantic search pipelines to retrieve relevant information and augment reviews with context for increased accuracy and developer support.

- *Outcomes:* The field experiment demonstrated that AI-led, upfront reviews were generally preferred by developers, although acceptance depended on the reviewer's familiarity with codebases and pull request severity. Key benefits included reduced context switching and improved review efficiency, though concerns about false positives and

trust in the AI were noted. The approach showed promise for enhanced developer experience and workflow optimization.

3.3 Enhanced Code Generation via Multi-Stage Performance-Guided LLM Orchestration

- *Data:* This work conducted a comprehensive empirical study analyzing the performance of 17 state-of-the-art LLMs on the HumanEval-X benchmark across five programming languages (Python, Java, C++, Go, Rust). The dataset included various algorithmic domains and development stages, evaluated with functional correctness and runtime metrics such as execution time and memory usage.
- *Algorithm:* The authors introduced PerfOrch, a multi-stage LLM orchestration framework that dynamically routes code generation tasks to the most suitable LLMs following a generate-fix-refine workflow. It incorporates stage-wise validation and rollback mechanisms without requiring model fine-tuning. The system uses performance guidance based on empirical profiling to select LLMs for each task context.
- *Outcomes:* PerfOrch outperformed single-model baselines with correctness rates of 96.22% on HumanEval-X and 91.37% on EffiBench-X, surpassing GPT-4o's 78.66% and 49.11%. Additionally, it improved execution speeds for over half of the tested problems with median speedups of 17.67% to 27.66%, demonstrating both correctness and efficiency improvements. The framework's scalability and plug-and-play design allow continual integration of new models.

3.4 Enhancing State Awareness for Multi-Step LLM Agent Tasks

- *Data:* The proposed Task Memory Engine (TME) was evaluated through case studies and comparative experiments on multi-step agent tasks, with datasets involving hierarchical task structures and sub-task relationships to test multi-step reasoning and execution consistency.
- *Algorithm:* TME is a lightweight, structured memory module that employs a hierar-

chical Task Memory Tree (TMT). Each node corresponds to a task step and stores input, output, status, and sub-task relationships. A prompt synthesis method dynamically generates LLM prompts based on the active node path to maintain contextual grounding and improve task execution coherence over multiple steps.

- *Outcomes:* TME led to better task completion accuracy, more interpretable agent behavior, and reduced hallucinations in multi-step LLM applications with minimal implementation overhead. The structured memory improved long-range coherence and robustness of autonomous LLM agents in executing complex, multi-step tasks.

This summary provides an overview of the data sources, core algorithms, and key results from these recent works on intelligent software assistants and LLM-based programming support systems.

4 Investigated Approaches

4.1 Document Upload

4.1.1 Data Indexing and Chunking

Importance of Chunking

Chunking constitutes a central preprocessing operation in Retrieval-Augmented Generation (RAG) pipelines. Its primary purpose is to partition long documents into smaller, semantically coherent segments that can be efficiently embedded, stored, retrieved, and interpreted by downstream language models. Chunking directly addresses several constraints of modern LLM-based architectures:

- **Token Limit Constraints:** Large documents frequently exceed the context windows of modern LLMs. Chunking mitigates this issue by enabling processing in smaller, context-preserving units.

- **Retrieval Accuracy:** Retrieval models (e.g., dense vector search, cosine similarity) perform significantly better when the indexed units contain a single, focused concept rather than multi-topic content.
- **Context Preservation:** Maintaining local semantic coherence ensures that retrieved chunks provide relevant and meaningful information during generation.
- **Computational Efficiency:** Smaller, well-defined chunks reduce embedding computation time and memory consumption.
- **Flexibility Across Document Types:** Technical documents, policies, rule sets, and legal texts vary in structure; chunking must adapt accordingly.

Overall, chunking plays the role of an intermediary layer that bridges data retrieval and generative reasoning, ensuring accurate, traceable, and contextually grounded results. Without an adequate chunking strategy, RAG-based systems tend to hallucinate, lose context, or return irrelevant results.

Types of Chunking

Fixed-Size Chunking

A method in which the text is divided into uniform units based on a predefined token, word, or character count.

Advantages: Simple to implement, scalable, predictable behavior.
Disadvantages: High risk of semantic fragmentation; unrelated information may accidentally be merged, thereby harming retrieval relevance.

Sentence-Based Chunking

The document is split according to natural sentence boundaries, optionally grouping consecutive sentences.

Advantages: Preserves grammatical and semantic flow; improves relevance of retrieved chunks.

Disadvantages: Chunk sizes vary significantly; additional logic is required for grouping sentences.

Semantic-Based Chunking

The text is segmented based on semantic similarity or topic coherence, typically using cosine similarity between sentence embeddings.

Advantages: Strongest preservation of semantic boundaries; high-quality retrieval performance; adaptable to heterogeneous document structures.
Disadvantages: Computationally more expensive; requires embedding inference during chunking.

Document-Specific Chunking

Chunking logic is adapted to the document's intrinsic structure (e.g., headings, numbered lists, articles, subclauses, or tables).

Advantages: Excellent contextual fidelity; ideal for legal, policy, and regulatory documents.
Disadvantages: Complex preprocessing requirements; dependent on accurate structure detection.

Agentic Chunking

Chunking designed around the operational tasks that an AI agent must perform (e.g., classification, retrieval, compliance checking).

Advantages: Task-specific optimization, enhanced interpretability, suitable for multi-agent frameworks.
Disadvantages: Requires additional modeling and task understanding; harder to generalize.

4.1.2 Embedding

Embeddings encode text into high-dimensional vector representations that preserve semantic relationships. These vector representations are fundamental for similarity search, contextual retrieval, clustering, and semantic indexing.

Types of Embedding Models

Word-Level Models

Examples: Word2Vec, GloVe, FastText.
Assign fixed vectors to words irrespective of context, useful for simple tasks, limited for nuanced semantic interpretation.

Contextual Embedding Models

Examples: BERT, RoBERTa, sentence-transformers.
Embeddings vary according to surrounding context, enabling deeper semantic understanding.

Sentence and Document Embedding Models

Examples: Sentence-BERT, E5, OpenAI embeddings.
Generate embeddings for entire sentences or paragraphs, making them highly suitable for retrieval tasks involving long-form documents.

4.1.3 Our Approach

Architecture Overview

The implemented pipeline integrates several components designed to convert raw rule documents into structured compliance evaluations. The workflow proceeds as follows:

Design Document (PDF/DOCX/TXT)

↓ Extract raw text

Semantic Chunker (MiniLM-L6-v2, threshold 0.7)

↓ Generate chunks and embeddings

Document Retriever (Cosine Similarity)

↓ Retrieve Top-K most relevant chunks

Reasoner (Llama 3.1-70B Instruct)

↓ Structured JSON response

Compliance Result (violations, explanations, fixes)

The architecture is optimized for rule documents, policy compliance tasks, and code-to-policy alignment scenarios. Each stage is modular, allowing future improvements (e.g., alternative embedding models, multi-agent orchestration, or hybrid symbolic–neural reasoning models).

Chunking Strategy

The system employs a **semantic-based chunking strategy**. The segmentation pipeline operates as follows:

1. The input document is first split into individual sentences.
2. Each sentence is embedded using the all-MiniLM-L6-v2 sentence transformer.
3. Cosine similarity is computed between consecutive sentence embeddings.
4. Sentences are grouped into chunks if their semantic similarity exceeds a selected threshold.
5. Each resulting chunk forms a self-contained rule, requirement, or policy element.

Multiple thresholds were experimentally evaluated to determine the optimal configuration. The results are summarized below:

Threshold	Observations
0.3	Produced excessively broad chunks; many unrelated concepts merged.
0.5	Improved grouping but still merged large sections of design documents.
0.7	**Optimal balance: coherent chunks of manageable size; highest retrieval quality.**
0.9	Too strict; resulted in excessively fragmented chunks.

As shown in Figure 1, the threshold of 0.7 ensures that each chunk is semantically meaningful and appropriate for later retrieval by the compliance engine.

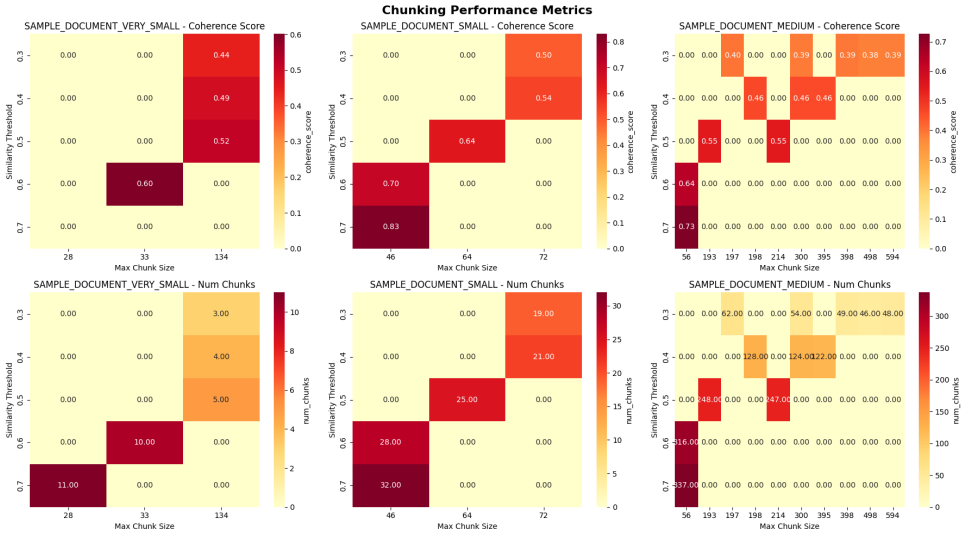

Figure 1: Threshold and maximum chunking tests

Embedding Model Selection

The system uses the **all-MiniLM-L6-v2** embedding model, selected due to its performance–efficiency tradeoff and suitability for local execution. Its main characteristics are:

- **Model size:** 22.7M parameters (∼80 MB)

- **Embedding dimensionality:** 384
- **Training data:** Approximately 1 billion sentence pairs, including datasets such as Reddit (2015–2018), S2ORC citation pairs, and WikiAnswers.
- **Execution environment:** Fully local, ensuring privacy and avoiding vendor lock-in.

This model provides robust semantic discrimination while maintaining fast inference times, making it ideal for compliance workflows involving frequent document updates.

Reasoner: LLM-Based Compliance Engine

To evaluate compliance between code snippets and document rules, the system uses the **Llama 3.1-70B Instruct** model hosted on the Hugging Face Inference API. The model performs the following tasks:

- Interpreting rules extracted from the uploaded document.
- Comparing them to a given code sample.
- Identifying violations or missing implementations.
- Generating human-readable explanations and potential remediation steps.

Model specifications:

- **Parameters:** 70 billion
- **Training data:** Instruction datasets from books, academic corpora, technical specifications, and over 25 million synthetic examples.
- **Output format:** A structured JSON object for programmatic consumption.

Example Output

```
{
  "compliant": false,
  "violations": [
    { "rule": "...", "description": "...", "fix": "..." }
  ],
  "missing_implementations": [],
  "summary": "..."
}
```

5 Multi-Agent System Architecture

This section presents the design, responsibilities, and coordination logic of the multi-agent orchestration framework implemented in the system. The architecture adopts a distributed model in which four specialized agents — the *Main Agent*, *Architecture Agent*, *Builder Agent*, and *Review Agent* — collaborate to process user requests, generate code, validate software architecture, and perform quality assurance tasks. Each component operates under strict behavioral and communication constraints, enabling a reliable and deterministic pipeline.

The agents communicate exclusively through structured JSON messages, ensuring unambiguous, machine-readable interactions suitable for automated pipelines or scalable multi-model execution.

5.1 Agent Overview

5.1.1 Main Agent

The Main Agent serves as the orchestrator and central decision-making component of the system. It is the only agent permitted to communicate with the user directly. Its

responsibilities include:

- Interpreting the user's intent and request semantics.
- Determining whether the request requires the full multi-agent development pipeline (Flow 1) or a direct non-code response (Flow 2).
- Delegating tasks to the Architecture, Builder, or Review agents when appropriate.
- Managing workflow ordering, context propagation, and JSON message formatting.
- Ensuring that architectural design, code generation, and review steps occur in the correct sequence.

The Main Agent does **not** generate, evaluate, or modify code. Instead, it acts as a controller that routes tasks to more specialized agents. It ensures that architectural validation precedes code generation and that code is always reviewed before being communicated back to the user.

Communication Behavior

The Main Agent produces three types of JSON messages:

1. **Delegation Messages** sent to Architecture, Builder, or Review agents.
2. **User Responses** when the output is final or when Flow 2 is selected.
3. **File Requests** when additional project files are required to continue the workflow.

5.1.2 Architecture Agent

The Architecture Agent is responsible for evaluating and refactoring code from a structural and architectural perspective. Its role focuses on high-level software engineering principles, including modularity, maintainability, design patterns, and adherence to SOLID principles.

Key responsibilities include:

- Detecting architectural anti-patterns such as cyclic dependencies, tight coupling, or god classes.
- Enforcing separation of concerns, clean boundaries, and correct layering.
- Correcting or refining class design, dependency direction, and module organization.
- Refactoring code when structural improvements are necessary, while preserving functionality.

The Architecture Agent outputs:

- A full corrected file (never partial or diff-based).
- A detailed list of architectural fixes.
- A summary of the refactoring performed.
- A routing directive (`"next_node"`) indicating whether execution should proceed to the Builder Agent.

Communication Behavior

The Architecture Agent sends a JSON message back to the Main Agent, which then forwards the output to the Builder Agent for implementation of structural changes in new or existing files.

5.1.3 Builder Agent

The Builder Agent is the code implementation model responsible for generating and modifying program files based on instructions received from the Architecture Agent. It is a deterministic code generator with the following capabilities:

- Creating new files when specified.
- Modifying existing files while preserving required functionality.

- Applying reviewer feedback precisely and without deviation.
- Producing complete file content for all modified components.

The Builder Agent outputs a structured JSON object that includes:

- Fully generated or modified files.
- Optional dependencies introduced by the new implementation.
- Setup instructions when required.
- A task list for the Review Agent.

Communication Behavior

The Builder Agent always forwards its output to the Review Agent. No direct communication with the user or the Architecture Agent occurs.

5.1.4 Review Agent

The Review Agent functions as a readability and style correctness evaluator. It ensures that the generated code adheres to standard software quality guidelines related to:

- Readability, naming clarity, and consistent formatting.
- Removal of dead, redundant, or confusing code.
- Simplification of expressions or refactoring long methods.
- Ensuring code follows human-centered readability principles.

The Review Agent acts like a linter and applies only non-functional improvements. It must return:

- A full, corrected version of the file.

- A list of readability and style improvements.
- A short summary of applied changes.
- A `"next_node"` directive, which is always set to `"none"`.

5.2 Inter-Agent Communication Protocol

Communication between agents is strictly governed by JSON schemas. Outputs from one agent act as inputs for the next, ensuring deterministic workflows. The communication takes place exclusively in the following direction:

Main → Architecture → Builder → Review → Main

Each agent returns structured outputs that must be:

- Valid JSON.
- Complete and self-contained.
- Suitable for direct machine parsing.
- Free of commentary, markdown, or partial information.

5.3 Workflow Models

5.3.1 Flow 1: Full Development Pipeline

Flow 1 is executed when the user's request requires architectural reasoning, code generation, modification, extension, debugging, or any form of code analysis. This workflow consists of the following stages:

1. **User Request** reaches the Main Agent.
2. The Main Agent determines that architectural or code tasks are required.

3. The request is delegated to the Architecture Agent for structural analysis.
4. The Architecture Agent refactors the file and forwards its output to the Builder Agent.
5. The Builder Agent generates code and sends it to the Review Agent.
6. A loop may occur:

   Builder ↔ Review

   until the Review Agent is satisfied.
7. The Main Agent validates pipeline completion and responds to the user.

5.3.2 Flow 2: Direct Response Pipeline

Flow 2 applies to requests that require conceptual explanations, documentation, or non-programming answers. The workflow is:

User → Main Agent → User

No delegation to other agents occurs.

5.4 System Behavior and Determinism

The system is designed to be deterministic and reproducible. Core behavioral rules include:

- Agents must never perform tasks outside their specialization (e.g., Main Agent writing code).
- No agent may produce incomplete outputs.
- All file outputs must be complete and rewritten in full.

- Code must always be reviewed before being presented to the user.
- The Main Agent must select the correct workflow based on intent detection.

5.5 Conclusion

The multi-agent architecture provides a modular, reliable, and robust pipeline for generating high-quality software. Each agent specializes in a well-defined aspect of the software engineering lifecycle, and through strict communication protocols, the system ensures architectural correctness, production-quality code, and human-readable output. The specialization and cooperation of all four agents allow the system to scale to complex tasks while maintaining clarity, determinism, and correctness.